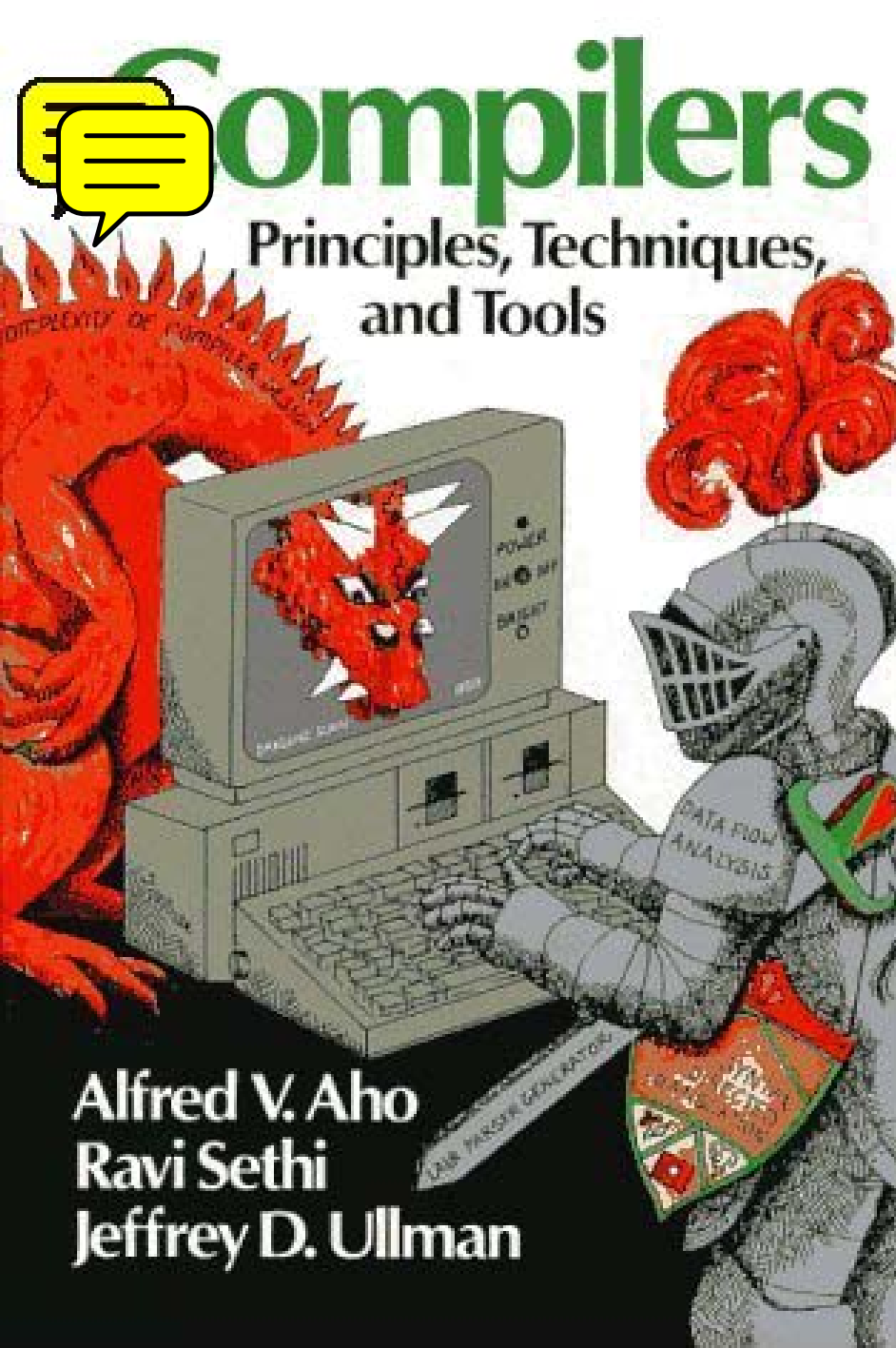
Compilers

Principles, Techniques, and Tools



Alfred V. Aho
Ravi Sethi
Jeffrey D. Ullman

Preface

This book is a descendant of *Principles of Compiler Design* by Alfred V. Aho and Jeffrey D. Ullman. Like its ancestor, it is intended as a text for a first course in compiler design. The emphasis is on solving problems universally encountered in designing a language translator, regardless of the source or target machine.

Although few people are likely to build or even maintain a compiler for a major programming language, the reader can profitably apply the ideas and techniques discussed in this book to general software design. For example, the string matching techniques for building lexical analyzers have also been used in text editors, information retrieval systems, and pattern recognition programs. Context-free grammars and syntax-directed definitions have been used to build many little languages such as the typesetting and figure drawing systems that produced this book. The techniques of code optimization have been used in program verifiers and in programs that produce "structured" programs from unstructured ones.

Use of the Book

The major topics in compiler design are covered in depth. The first chapter introduces the basic structure of a compiler and is essential to the rest of the book.

Chapter 2 presents a translator from infix to postfix expressions, built using some of the basic techniques described in this book. Many of the remaining chapters amplify the material in Chapter 2.

Chapter 3 covers lexical analysis, regular expressions, finite-state machines, and scanner-generator tools. The material in this chapter is broadly applicable to text-processing.

Chapter 4 covers the major parsing techniques in depth, ranging from the recursive-descent methods that are suitable for hand implementation to the computationally more intensive LR techniques that have been used in parser generators.

Chapter 5 introduces the principal ideas in syntax-directed translation. This chapter is used in the remainder of the book for both specifying and implementing translations.

Chapter 6 presents the main ideas for performing static semantic checking. Type checking and unification are discussed in detail.

Chapter 7 discusses storage organizations used to support the run-time environment of a program.

Chapter 8 begins with a discussion of intermediate languages and then shows how common programming language constructs can be translated into intermediate code.

Chapter 9 covers target code generation. Included are the basic "on-the-fly" code generation methods, as well as optimal methods for generating code for expressions. Peephole optimization and code-generator generators are also covered.

Chapter 10 is a comprehensive treatment of code optimization. Data-flow analysis methods are covered in detail, as well as the principal methods for global optimization.

Chapter 11 discusses some pragmatic issues that arise in implementing a compiler. Software engineering and testing are particularly important in compiler construction.

Chapter 12 presents case studies of compilers that have been constructed using some of the techniques presented in this book.

Appendix A describes a simple language, a "subset" of Pascal, that can be used as the basis of an implementation project.

The authors have taught both introductory and advanced courses, at the undergraduate and graduate levels, from the material in this book at AT&T Bell Laboratories, Columbia, Princeton, and Stanford.

An introductory compiler course might cover material from the following sections of this book:

introduction	Chapter 1 and Sections 2.1-2.5
lexical analysis	2.6, 3.1-3.4
symbol tables	2.7, 7.6
parsing	2.4, 4.1-4.4
syntax-directed translation	2.5, 5.1-5.5
type checking	6.1-6.2
run-time organization	7.1-7.3
intermediate code generation	8.1-8.3
code generation	9.1-9.4
code optimization	10.1-10.2

Information needed for a programming project like the one in Appendix A is introduced in Chapter 2.

A course stressing tools in compiler construction might include the discussion of lexical analyzer generators in Sections 3.5, of parser generators in Sections 4.8 and 4.9, of code-generator generators in Section 9.12, and material on techniques for compiler construction from Chapter 11.

An advanced course might stress the algorithms used in lexical analyzer generators and parser generators discussed in Chapters 3 and 4, the material

on type equivalence, overloading, polymorphism, and unification in Chapter 6, the material on run-time storage organization in Chapter 7, the pattern-directed code generation methods discussed in Chapter 9, and material on code optimization from Chapter 10.

Exercises

As before, we rate exercises with stars. Exercises without stars test understanding of definitions, singly starred exercises are intended for more advanced courses, and doubly starred exercises are food for thought.

Acknowledgments

At various stages in the writing of this book, a number of people have given us invaluable comments on the manuscript. In this regard we owe a debt of gratitude to Bill Appelbe, Nelson Beebe, Jon Bentley, Lois Bogess, Rodney Farrow, Stu Feldman, Charles Fischer, Chris Fraser, Art Gittelman, Eric Grosse, Dave Hanson, Fritz Henglein, Robert Henry, Gerard Holzmann, Steve Johnson, Brian Kernighan, Ken Kubota, Daniel Lehmann, Dave MacQueen, Dianne Maki, Alan Martin, Doug McIlroy, Charles McLaughlin, John Mitchell, Elliott Organick, Robert Paige, Phil Pfeiffer, Rob Pike, Kari-Jouko Räihä, Dennis Ritchie, Sriram Sankar, Paul Stoecker, Bjarne Stroustrup, Tom Szymanski, Kim Tracy, Peter Weinberger, Jennifer Widom, and Reinhard Wilhelm.

This book was phototypeset by the authors using the excellent software available on the UNIX system. The typesetting command read

```
pic files | tbl | eqn | troff -ms
```

`pic` is Brian Kernighan's language for typesetting figures; we owe Brian a special debt of gratitude for accommodating our special and extensive figure-drawing needs so cheerfully. `tbl` is Mike Lesk's language for laying out tables. `eqn` is Brian Kernighan and Lorinda Cherry's language for typesetting mathematics. `troff` is Joe Ossana's program for formatting text for a phototypesetter, which in our case was a Mergenthaler Linotron 202/N. The `ms` package of `troff` macros was written by Mike Lesk. In addition, we managed the text using `make` due to Stu Feldman. Cross references within the text were maintained using `awk` created by Al Aho, Brian Kernighan, and Peter Weinberger, and `sed` created by Lee McMahon.

The authors would particularly like to acknowledge Patricia Solomon for helping prepare the manuscript for photocomposition. Her cheerfulness and expert typing were greatly appreciated. J. D. Ullman was supported by an Einstein Fellowship of the Israeli Academy of Arts and Sciences during part of the time in which this book was written. Finally, the authors would like to thank AT&T Bell Laboratories for its support during the preparation of the manuscript.

A. V. A., R. S., J. D. U.

Contents

CHAPTER 1

Introduction to Compiling

The principles and techniques of compiler writing are so pervasive that the ideas found in this book will be used many times in the career of a computer scientist. Compiler writing spans programming languages, machine architecture, language theory, algorithms, and software engineering. Fortunately, a few basic compiler-writing techniques can be used to construct translators for a wide variety of languages and machines. In this chapter, we introduce the subject of compiling by describing the components of a compiler, the environment in which compilers do their job, and some software tools that make it easier to build compilers.

1.1 COMPILERS

Simply stated, a compiler is a program that reads a program written in one language – the *source* language – and translates it into an equivalent program in another language – the *target* language (see Fig. 1.1). As an important part of this translation process, the compiler reports to its user the presence of errors in the source program.

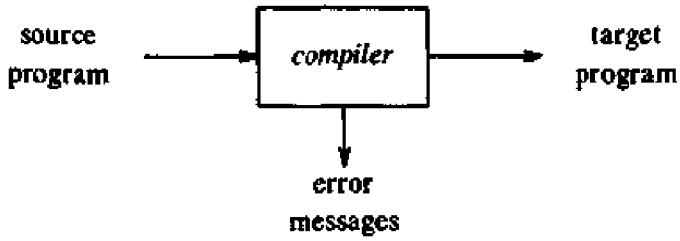


Fig. 1.1. A compiler.

At first glance, the variety of compilers may appear overwhelming. There are thousands of source languages, ranging from traditional programming languages such as Fortran and Pascal to specialized languages that have arisen in virtually every area of computer application. Target languages are equally as varied; a target language may be another programming language, or the machine language of any computer between a microprocessor and a

supercomputer. Compilers are sometimes classified as single-pass, multi-pass, load-and-go, debugging, or optimizing, depending on how they have been constructed or on what function they are supposed to perform. Despite this apparent complexity, the basic tasks that any compiler must perform are essentially the same. By understanding these tasks, we can construct compilers for a wide variety of source languages and target machines using the same basic techniques.

Our knowledge about how to organize and write compilers has increased vastly since the first compilers started to appear in the early 1950's. It is difficult to give an exact date for the first compiler because initially a great deal of experimentation and implementation was done independently by several groups. Much of the early work on compiling dealt with the translation of arithmetic formulas into machine code.

Throughout the 1950's, compilers were considered notoriously difficult programs to write. The first Fortran compiler, for example, took 18 staff-years to implement (Backus et al. [1957]). We have since discovered systematic techniques for handling many of the important tasks that occur during compilation. Good implementation languages, programming environments, and software tools have also been developed. With these advances, a substantial compiler can be implemented even as a student project in a one-semester compiler-design course.

The Analysis-Synthesis Model of Compilation

There are two parts to compilation: analysis and synthesis. The analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program. The synthesis part constructs the desired target program from the intermediate representation. Of the two parts, synthesis requires the most specialized techniques. We shall consider analysis informally in Section 1.2 and outline the way target code is synthesized in a standard compiler in Section 1.3.

During analysis, the operations implied by the source program are determined and recorded in a hierarchical structure called a tree. Often, a special kind of tree called a syntax tree is used, in which each node represents an operation and the children of a node represent the arguments of the operation. For example, a syntax tree for an assignment statement is shown in Fig. 1.2.

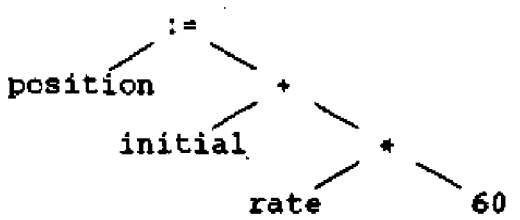


Fig. 1.2. Syntax tree for `position := initial + rate * 60`.

Many software tools that manipulate source programs first perform some kind of analysis. Some examples of such tools include:

1. *Structure editors.* A structure editor takes as input a sequence of commands to build a source program. The structure editor not only performs the text-creation and modification functions of an ordinary text editor, but it also analyzes the program text, putting an appropriate hierarchical structure on the source program. Thus, the structure editor can perform additional tasks that are useful in the preparation of programs. For example, it can check that the input is correctly formed, can supply keywords automatically (e.g., when the user types `while`, the editor supplies the matching `do` and reminds the user that a conditional must come between them), and can jump from a `begin` or left parenthesis to its matching `end` or right parenthesis. Further, the output of such an editor is often similar to the output of the analysis phase of a compiler.

2. *Pretty printers.* A pretty printer analyzes a program and prints it in such a way that the structure of the program becomes clearly visible. For example, comments may appear in a special font, and statements may appear with an amount of indentation proportional to the depth of their nesting in the hierarchical organization of the statements.

3. *Static checkers.* A static checker reads a program, analyzes it, and attempts to discover potential bugs without running the program. The analysis portion is often similar to that found in optimizing compilers of the type discussed in Chapter 10. For example, a static checker may detect that parts of the source program can never be executed, or that a certain variable might be used before being defined. In addition, it can catch logical errors such as trying to use a real variable as a pointer, employing the type-checking techniques discussed in Chapter 6.

4. *Interpreters.* Instead of producing a target program as a translation, an interpreter performs the operations implied by the source program. For an assignment statement, for example, an interpreter might build a tree like Fig. 1.2, and then carry out the operations at the nodes as it "walks" the tree. At the root it would discover it had an assignment to perform, so it would call a routine to evaluate the expression on the right, and then store the resulting value in the location associated with the identifier `position`. At the right child of the root, the routine would discover it had to compute the sum of two expressions. It would call itself recursively to compute the value of the expression `rate*60`. It would then add that value to the value of the variable `initial`.

   Interpreters are frequently used to execute command languages, since each operator executed in a command language is usually an invocation of a complex routine such as an editor or compiler. Similarly, some "very high-level" languages, like APL, are normally interpreted because there are many things about the data, such as the size and shape of arrays, that

cannot be deduced at compile time.

Traditionally, we think of a compiler as a program that translates a source language like Fortran into the assembly or machine language of some computer. However, there are seemingly unrelated places where compiler technology is regularly used. The analysis portion in each of the following examples is similar to that of a conventional compiler.

1. *Text formatters.* A text formatter takes input that is a stream of characters, most of which is text to be typeset, but some of which includes commands to indicate paragraphs, figures, or mathematical structures like subscripts and superscripts. We mention some of the analysis done by text formatters in the next section.

2. *Silicon compilers.* A silicon compiler has a source language that is similar or identical to a conventional programming language. However, the variables of the language represent, not locations in memory, but, logical signals (0 or 1) or groups of signals in a switching circuit. The output is a circuit design in an appropriate language. See Johnson [1983], Ullman [1984], or Trickey [1985] for a discussion of silicon compilation.

3. *Query interpreters.* A query interpreter translates a predicate containing relational and boolean operators into commands to search a database for records satisfying that predicate. (See Ullman [1982] or Date [1986].)

The Context of a Compiler

In addition to a compiler, several other programs may be required to create an executable target program. A source program may be divided into modules stored in separate files. The task of collecting the source program is sometimes entrusted to a distinct program, called a preprocessor. The preprocessor may also expand shorthands, called macros, into source language statements.

Figure 1.3 shows a typical "compilation." The target program created by the compiler may require further processing before it can be run. The compiler in Fig. 1.3 creates assembly code that is translated by an assembler into machine code and then linked together with some library routines into the code that actually runs on the machine.

We shall consider the components of a compiler in the next two sections; the remaining programs in Fig. 1.3 are discussed in Section 1.4.

1.2 ANALYSIS OF THE SOURCE PROGRAM

In this section, we introduce analysis and illustrate its use in some text-formatting languages. The subject is treated in more detail in Chapters 2-4 and 6. In compiling, analysis consists of three phases:

1. *Linear analysis*, in which the stream of characters making up the source program is read from left-to-right and grouped into *tokens* that are sequences of characters having a collective meaning.

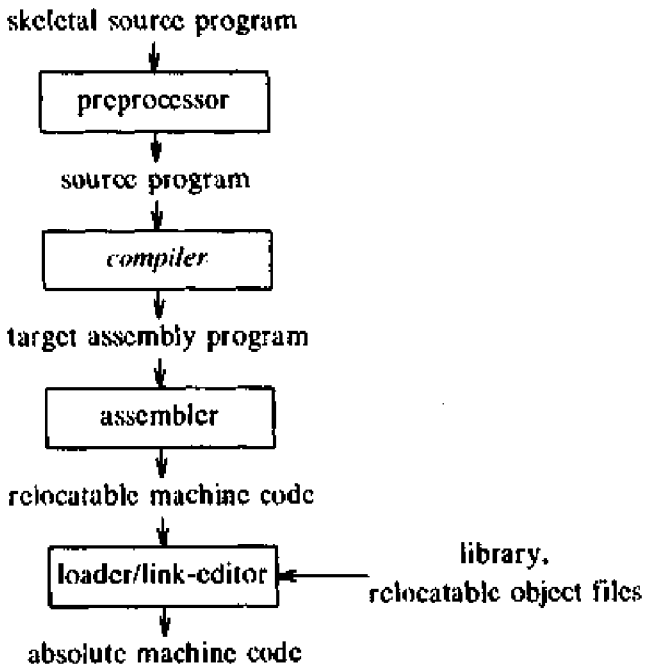


Fig. 1.3. A language-processing system.

2. *Hierarchical analysis*, in which characters or tokens are grouped hierarchically into nested collections with collective meaning.
3. *Semantic analysis*, in which certain checks are performed to ensure that the components of a program fit together meaningfully.

Lexical Analysis

In a compiler, linear analysis is called *lexical analysis* or *scanning*. For example, in lexical analysis the characters in the assignment statement

```
position := initial + rate * 60
```

would be grouped into the following tokens:

1. The identifier `position`.
2. The assignment symbol `:=`.
3. The identifier `initial`.
4. The plus sign.
5. The identifier `rate`.
6. The multiplication sign.
7. The number 60.

The blanks separating the characters of these tokens would normally be eliminated during lexical analysis.

Syntax Analysis

Hierarchical analysis is called *parsing* or *syntax analysis*. It involves grouping the tokens of the source program into grammatical phrases that are used by the compiler to synthesize output. Usually, the grammatical phrases of the source program are represented by a parse tree such as the one shown in Fig. 1.4.

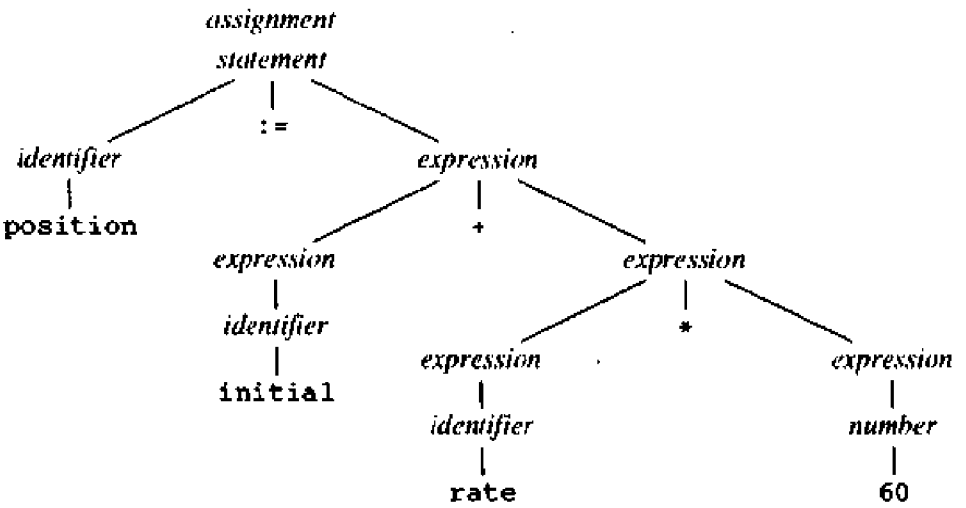


Fig. 1.4. Parse tree for `position := initial + rate * 60`.

In the expression `initial + rate * 60`, the phrase `rate * 60` is a logical unit because the usual conventions of arithmetic expressions tell us that multiplication is performed before addition. Because the expression `initial + rate` is followed by a `*`, it is not grouped into a single phrase by itself in Fig. 1.4.

The hierarchical structure of a program is usually expressed by recursive rules. For example, we might have the following rules as part of the definition of expressions:

1. Any *identifier* is an expression.
2. Any *number* is an expression.
3. If $expression_1$ and $expression_2$ are expressions, then so are

$$expression_1 + expression_2$$
$$expression_1 * expression_2$$
$$(expression_1)$$

Rules (1) and (2) are (nonrecursive) basis rules, while (3) defines expressions in terms of operators applied to other expressions. Thus, by rule (1), `initial` and `rate` are expressions. By rule (2), `60` is an expression, while by rule (3), we can first infer that `rate * 60` is an expression and finally that `initial + rate * 60` is an expression.

Similarly, many languages define statements recursively by rules such as:

1. If *identifier*$_1$ is an identifier, and *expression*$_2$ is an expression, then

   identifier$_1$:= *expression*$_2$

   is a statement.

2. If *expression*$_1$ is an expression and *statement*$_2$ is a statement, then

   while (*expression*$_1$) **do** *statement*$_2$
   if (*expression*$_1$) **then** *statement*$_2$

   are statements.

The division between lexical and syntactic analysis is somewhat arbitrary. We usually choose a division that simplifies the overall task of analysis. One factor in determining the division is whether a source language construct is inherently recursive or not. Lexical constructs do not require recursion, while syntactic constructs often do. Context-free grammars are a formalization of recursive rules that can be used to guide syntactic analysis. They are introduced in Chapter 2 and studied extensively in Chapter 4.

For example, recursion is not required to recognize identifiers, which are typically strings of letters and digits beginning with a letter. We would normally recognize identifiers by a simple scan of the input stream, waiting until a character that was neither a letter nor a digit was found, and then grouping all the letters and digits found up to that point into an identifier token. The characters so grouped are recorded in a table, called a symbol table, and removed from the input so that processing of the next token can begin.

On the other hand, this kind of linear scan is not powerful enough to analyze expressions or statements. For example, we cannot properly match parentheses in expressions, or `begin` and `end` in statements, without putting some kind of hierarchical or nesting structure on the input.

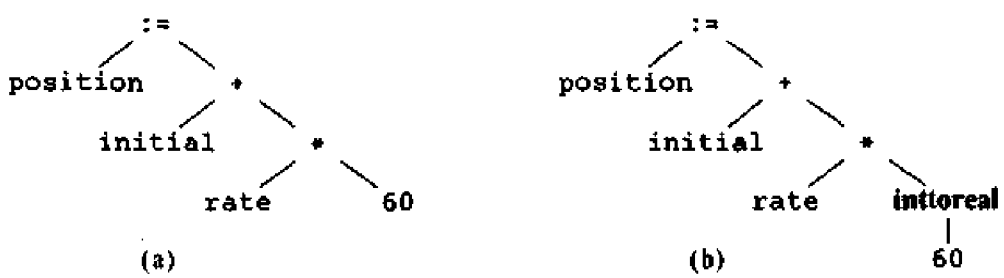


Fig. 1.5. Semantic analysis inserts a conversion from integer to real.

The parse tree in Fig. 1.4 describes the syntactic structure of the input. A more common internal representation of this syntactic structure is given by the syntax tree in Fig. 1.5(a). A syntax tree is a compressed representation of the parse tree in which the operators appear as the interior nodes, and the operands of an operator are the children of the node for that operator. The construction of trees such as the one in Fig. 1.5(a) is discussed in Section 5.2.

We shall take up in Chapter 2, and in more detail in Chapter 5, the subject of *syntax-directed translation*, in which the compiler uses the hierarchical structure on the input to help generate the output.

Semantic Analysis

The semantic analysis phase checks the source program for semantic errors and gathers type information for the subsequent code-generation phase. It uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operands of expressions and statements.

An important component of semantic analysis is type checking. Here the compiler checks that each operator has operands that are permitted by the source language specification. For example, many programming language definitions require a compiler to report an error every time a real number is used to index an array. However, the language specification may permit some operand coercions, for example, when a binary arithmetic operator is applied to an integer and real. In this case, the compiler may need to convert the integer to a real. Type checking and semantic analysis are discussed in Chapter 6.

Example 1.1. Inside a machine, the bit pattern representing an integer is generally different from the bit pattern for a real, even if the integer and the real number happen to have the same value. Suppose, for example, that all identifiers in Fig. 1.5 have been declared to be reals and that 60 by itself is assumed to be an integer. Type checking of Fig. 1.5(a) reveals that * is applied to a real, `rate`, and an integer, 60. The general approach is to convert the integer into a real. This has been achieved in Fig. 1.5(b) by creating an extra node for the operator **inttoreal** that explicitly converts an integer into a real. Alternatively, since the operand of **inttoreal** is a constant, the compiler may instead replace the integer constant by an equivalent real constant. □

Analysis in Text Formatters

It is useful to regard the input to a text formatter as specifying a hierarchy of *boxes* that are rectangular regions to be filled by some bit pattern, representing light and dark pixels to be printed by the output device.

For example, the TEX system (Knuth [1984a]) views its input this way. Each character that is not part of a command represents a box containing the bit pattern for that character in the appropriate font and size. Consecutive characters not separated by "white space" (blanks or newline characters) are grouped into words, consisting of a sequence of horizontally arranged boxes, shown schematically in Fig. 1.6. The grouping of characters into words (or commands) is the linear or lexical aspect of analysis in a text formatter.

Boxes in TEX may be built from smaller boxes by arbitrary horizontal and vertical combinations. For example,

```
\hbox{ <list of boxes> }
```

Fig. 1.6. Grouping of characters and words into boxes.

groups the list of boxes by juxtaposing them horizontally, while the `\vbox` operator similarly groups a list of boxes by vertical juxtaposition. Thus, if we say in TEX

```
\hbox{\vbox{! 1} \vbox{@ 2}}
```

we get the arrangement of boxes shown in Fig. 1.7. Determining the hierarchical arrangement of boxes implied by the input is part of syntax analysis in TEX.

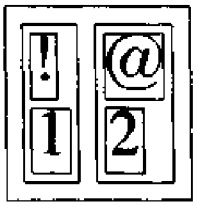


Fig. 1.7. Hierarchy of boxes in TEX.

As another example, the preprocessor EQN for mathematics (Kernighan and Cherry [1975]), or the mathematical processor in TEX, builds mathematical expressions from operators like `sub` and `sup` for subscripts and superscripts. If EQN encounters an input text of the form

BOX `sub` *box*

it shrinks the size of *box* and attaches it to *BOX* near the lower right corner, as illustrated in Fig. 1.8. The `sup` operator similarly attaches *box* at the upper right.

Fig. 1.8. Building the subscript structure in mathematical text.

These operators can be applied recursively, so, for example, the EQN text

```
a sub {i sup 2}
```

results in a_i^2. Grouping the operators **sub** and **sup** into tokens is part of the lexical analysis of EQN text. However, the syntactic structure of the text is needed to determine the size and placement of a box.

1.3 THE PHASES OF A COMPILER

Conceptually, a compiler operates in *phases*, each of which transforms the source program from one representation to another. A typical decomposition of a compiler is shown in Fig. 1.9. In practice, some of the phases may be grouped together, as mentioned in Section 1.5, and the intermediate representations between the grouped phases need not be explicitly constructed.

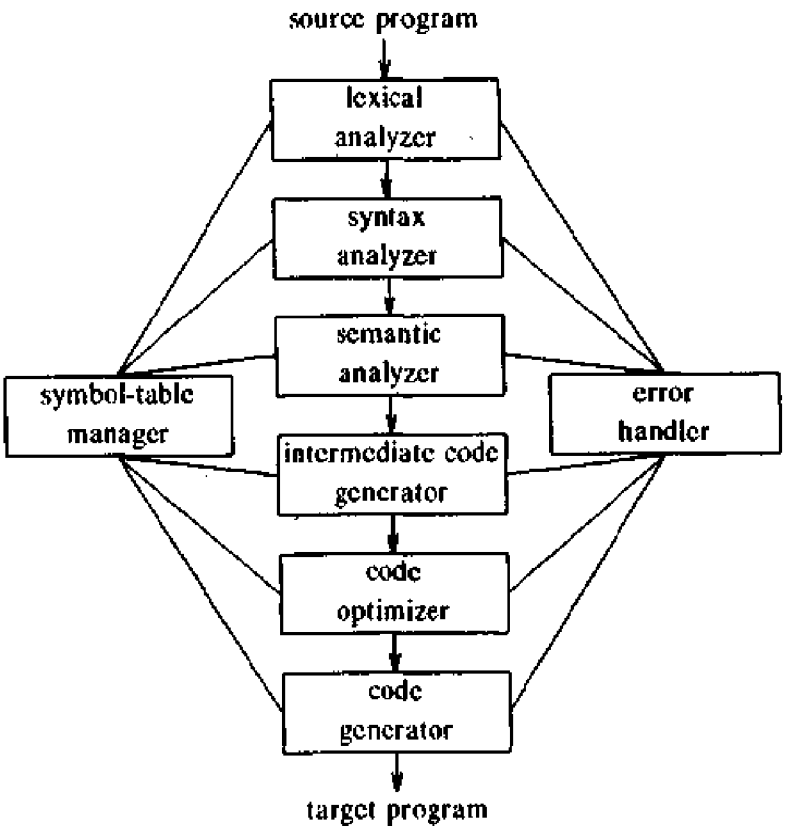


Fig. 1.9. Phases of a compiler.

The first three phases, forming the bulk of the analysis portion of a compiler, were introduced in the last section. Two other activities, symbol-table management and error handling, are shown interacting with the six phases of lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and code generation. Informally, we shall also call the symbol-table manager and the error handler "phases."

Symbol-Table Management

An essential function of a compiler is to record the identifiers used in the source program and collect information about various attributes of each identifier. These attributes may provide information about the storage allocated for an identifier, its type, its scope (where in the program it is valid), and, in the case of procedure names, such things as the number and types of its arguments, the method of passing each argument (e.g., by reference), and the type returned, if any.

A *symbol table* is a data structure containing a record for each identifier, with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly. Symbol tables are discussed in Chapters 2 and 7.

When an identifier in the source program is detected by the lexical analyzer, the identifier is entered into the symbol table. However, the attributes of an identifier cannot normally be determined during lexical analysis. For example, in a Pascal declaration like

```
var position, initial, rate : real ;
```

the type real is not known when `position`, `initial`, and `rate` are seen by the lexical analyzer.

The remaining phases enter information about identifiers into the symbol table and then use this information in various ways. For example, when doing semantic analysis and intermediate code generation, we need to know what the types of identifiers are, so we can check that the source program uses them in valid ways, and so that we can generate the proper operations on them. The code generator typically enters and uses detailed information about the storage assigned to identifiers.

Error Detection and Reporting

Each phase can encounter errors. However, after detecting an error, a phase must somehow deal with that error, so that compilation can proceed, allowing further errors in the source program to be detected. A compiler that stops when it finds the first error is not as helpful as it could be.

The syntax and semantic analysis phases usually handle a large fraction of the errors detectable by the compiler. The lexical phase can detect errors where the characters remaining in the input do not form any token of the language. Errors where the token stream violates the structure rules (syntax) of the language are determined by the syntax analysis phase. During semantic analysis the compiler tries to detect constructs that have the right syntactic structure but no meaning to the operation involved, e.g., if we try to add two identifiers, one of which is the name of an array, and the other the name of a procedure. We discuss the handling of errors by each phase in the part of the book devoted to that phase.

The Analysis Phases

As translation progresses, the compiler's internal representation of the source program changes. We illustrate these representations by considering the translation of the statement

```
position := initial + rate * 60          (1.1)
```

Figure 1.10 shows the representation of this statement after each phase.

The lexical analysis phase reads the characters in the source program and groups them into a stream of tokens in which each token represents a logically cohesive sequence of characters, such as an identifier, a keyword (`if`, `while`, etc.), a punctuation character, or a multi-character operator like `:=`. The character sequence forming a token is called the *lexeme* for the token.

Certain tokens will be augmented by a "lexical value." For example, when an identifier like `rate` is found, the lexical analyzer not only generates a token, say **id**, but also enters the lexeme `rate` into the symbol table, if it is not already there. The lexical value associated with this occurrence of **id** points to the symbol-table entry for `rate`.

In this section, we shall use $\mathbf{id}_1$, $\mathbf{id}_2$, and $\mathbf{id}_3$ for `position`, `initial`, and `rate`, respectively, to emphasize that the internal representation of an identifier is different from the character sequence forming the identifier. The representation of (1.1) after lexical analysis is therefore suggested by:

$$\mathbf{id}_1 := \mathbf{id}_2 + \mathbf{id}_3 * 60 \qquad (1.2)$$

We should also make up tokens for the multi-character operator `:=` and the number 60 to reflect their internal representation, but we defer that until Chapter 2. Lexical analysis is covered in detail in Chapter 3.

The second and third phases, syntax and semantic analysis, have also been introduced in Section 1.2. Syntax analysis imposes a hierarchical structure on the token stream, which we shall portray by syntax trees as in Fig. 1.11(a). A typical data structure for the tree is shown in Fig. 1.11(b) in which an interior node is a record with a field for the operator and two fields containing pointers to the records for the left and right children. A leaf is a record with two or more fields, one to identify the token at the leaf, and the others to record information about the token. Additional information about language constructs can be kept by adding more fields to the records for nodes. We discuss syntax and semantic analysis in Chapters 4 and 6, respectively.

Intermediate Code Generation

After syntax and semantic analysis, some compilers generate an explicit intermediate representation of the source program. We can think of this intermediate representation as a program for an abstract machine. This intermediate representation should have two important properties; it should be easy to produce, and easy to translate into the target program.

The intermediate representation can have a variety of forms. In Chapter 8,

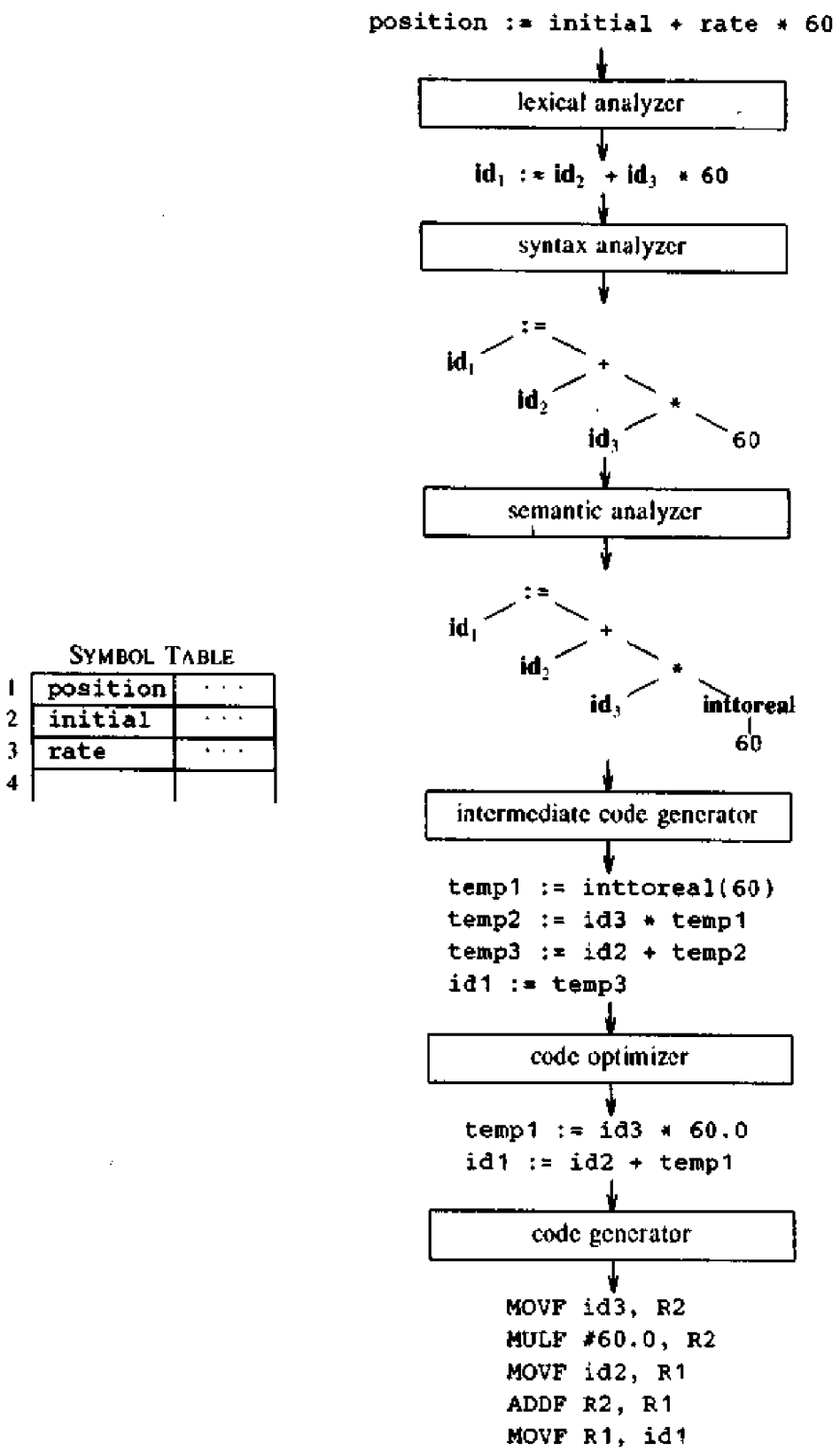


Fig. 1.10. Translation of a statement.

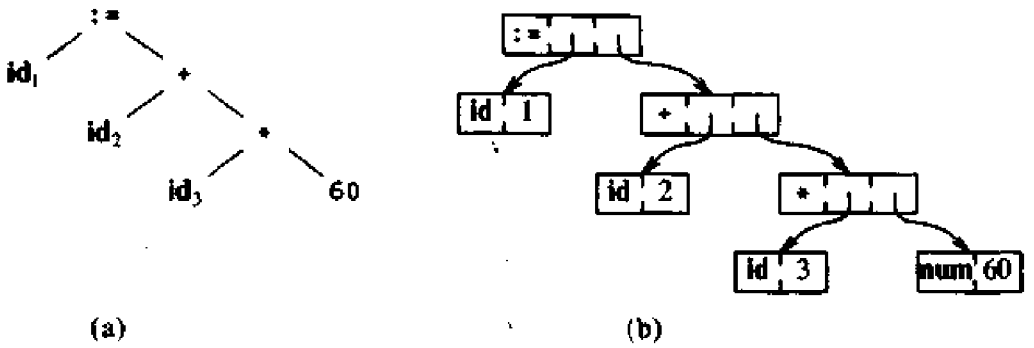


Fig. 1.11. The data structure in (b) is for the tree in (a).

we consider an intermediate form called "three-address code," which is like the assembly language for a machine in which every memory location can act like a register. Three-address code consists of a sequence of instructions, each of which has at most three operands. The source program in (1.1) might appear in three-address code as

```
temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```
(1.3)

This intermediate form has several properties. First, each three-address instruction has at most one operator in addition to the assignment. Thus, when generating these instructions, the compiler has to decide on the order in which operations are to be done; the multiplication precedes the addition in the source program of (1.1). Second, the compiler must generate a temporary name to hold the value computed by each instruction. Third, some "three-address" instructions have fewer than three operands, e.g., the first and last instructions in (1.3).

In Chapter 8, we cover the principal intermediate representations used in compilers. In general, these representations must do more than compute expressions; they must also handle flow-of-control constructs and procedure calls. Chapters 5 and 8 present algorithms for generating intermediate code for typical programming language constructs.

Code Optimization

The code optimization phase attempts to improve the intermediate code, so that faster-running machine code will result. Some optimizations are trivial. For example, a natural algorithm generates the intermediate code (1.3), using an instruction for each operator in the tree representation after semantic analysis, even though there is a better way to perform the same calculation, using the two instructions

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```
(1.4)

There is nothing wrong with this simple algorithm, since the problem can be fixed during the code-optimization phase. That is, the compiler can deduce that the conversion of 60 from integer to real representation can be done once and for all at compile time, so the `inttoreal` operation can be eliminated. Besides, `temp3` is used only once, to transmit its value to `id1`. It then becomes safe to substitute `id1` for `temp3`, whereupon the last statement of (1.3) is not needed and the code of (1.4) results.

There is great variation in the amount of code optimization different compilers perform. In those that do the most, called "optimizing compilers," a significant fraction of the time of the compiler is spent on this phase. However, there are simple optimizations that significantly improve the running time of the target program without slowing down compilation too much. Many of these are discussed in Chapter 9, while Chapter 10 gives the technology used by the most powerful optimizing compilers.

Code Generation

The final phase of the compiler is the generation of target code, consisting normally of relocatable machine code or assembly code. Memory locations are selected for each of the variables used by the program. Then, intermediate instructions are each translated into a sequence of machine instructions that perform the same task. A crucial aspect is the assignment of variables to registers.

For example, using registers 1 and 2, the translation of the code of (1.4) might become

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```
(1.5)

The first and second operands of each instruction specify a source and destination, respectively. The `F` in each instruction tells us that instructions deal with floating-point numbers. This code moves the contents of the address[1] `id3` into register 2, then multiplies it with the real-constant 60.0. The `#` signifies that `60.0` is to be treated as a constant. The third instruction moves `id2` into register 1 and adds to it the value previously computed in register 2. Finally, the value in register 1 is moved into the address of `id1`, so the code implements the assignment in Fig. 1.10. Chapter 9 covers code generation.

[1] We have side-stepped the important issue of storage allocation for the identifiers in the source program. As we shall see in Chapter 7, the organization of storage at run-time depends on the language being compiled. Storage-allocation decisions are made either during intermediate code generation or during code generation.

1.4 COUSINS OF THE COMPILER

As we saw in Fig. 1.3, the input to a compiler may be produced by one or more preprocessors, and further processing of the compiler's output may be needed before running machine code is obtained. In this section, we discuss the context in which a compiler typically operates.

Preprocessors

Preprocessors produce input to compilers. They may perform the following functions:

1. *Macro processing*. A preprocessor may allow a user to define macros that are shorthands for longer constructs.

2. *File inclusion*. A preprocessor may include header files into the program text. For example, the C preprocessor causes the contents of the file `<global.h>` to replace the statement `#include <global.h>` when it processes a file containing this statement.

3. *"Rational" preprocessors*. These processors augment older languages with more modern flow-of-control and data-structuring facilities. For example, such a preprocessor might provide the user with built-in macros for constructs like while-statements or if-statements, where none exist in the programming language itself.

4. *Language extensions*. These processors attempt to add capabilities to the language by what amounts to built-in macros. For example, the language Equel (Stonebraker et al. [1976]) is a database query language embedded in C. Statements beginning with ## are taken by the preprocessor to be database-access statements, unrelated to C, and are translated into procedure calls on routines that perform the database access.

Macro processors deal with two kinds of statement: macro definition and macro use. Definitions are normally indicated by some unique character or keyword, like `define` or `macro`. They consist of a name for the macro being defined and a *body*, forming its definition. Often, macro processors permit *formal parameters* in their definition, that is, symbols to be replaced by values (a "value" is a string of characters, in this context). The use of a macro consists of naming the macro and supplying *actual parameters*, that is, values for its formal parameters. The macro processor substitutes the actual parameters for the formal parameters in the body of the macro; the transformed body then replaces the macro use itself.

Example 1.2. The TEX typesetting system mentioned in Section 1.2 contains a general macro facility. Macro definitions take the form

`\define` <macro name> <template> {<body>}

A macro name is any string of letters preceded by a backslash. The template

is any string of characters, with strings of the form #1, #2, . . . , #9 regarded as formal parameters. These symbols may also appear in the body, any number of times. For example, the following macro defines a citation for the *Journal of the ACM*.

```
\define\JACM #1;#2;#3.
    {{\sl J. ACM} {\bf #1}:#2, pp. #3.}
```

The macro name is \JACM, and the template is "#1;#2;#3."; semicolons separate the parameters and the last parameter is followed by a period. A use of this macro must take the form of the template, except that arbitrary strings may be substituted for the formal parameters.[2] Thus, we may write

```
\JACM 17;4;715-728.
```

and expect to see

J. ACM **17**:4, pp. 715-728.

The portion of the body {\sl J. ACM} calls for an italicized ("slanted") "*J. ACM*". Expression {\bf #1} says that the first actual parameter is to be made boldface; this parameter is intended to be the volume number.

TEX allows any punctuation or string of text to separate the volume, issue, and page numbers in the definition of the \JACM macro. We could even have used no punctuation at all, in which case TEX would take each actual parameter to be a single character or a string surrounded by { } □

Assemblers

Some compilers produce assembly code, as in (1.5), that is passed to an assembler for further processing. Other compilers perform the job of the assembler, producing relocatable machine code that can be passed directly to the loader/link-editor. We assume the reader has some familiarity with what an assembly language looks like and what an assembler does; here we shall review the relationship between assembly and machine code.

Assembly code is a mnemonic version of machine code, in which names are used instead of binary codes for operations, and names are also given to memory addresses. A typical sequence of assembly instructions might be

```
MOV a, R1
ADD #2, R1                    (1.6)
MOV R1, b
```

This code moves the contents of the address a into register 1, then adds the constant 2 to it, treating the contents of register 1 as a fixed-point number,

[2] Well, almost arbitrary strings, since a simple left-to-right scan of the macro use is made, and as soon as a symbol matching the text following a *#i* symbol in the template is found, the preceding string is deemed to match *#i*. Thus, if we tried to substitute a;b;c;d for #1, we would find that only a matched #1 and b;c;d was matched to #2.

and finally stores the result in the location named by b. Thus, it computes b := a + 2.

It is customary for assembly languages to have macro facilities that are similar to those in the macro preprocessors discussed above.

Two-Pass Assembly

The simplest form of assembler makes two passes over the input, where a *pass* consists of reading an input file once. In the first pass, all the identifiers that denote storage locations are found and stored in a symbol table (separate from that of the compiler). Identifiers are assigned storage locations as they are encountered for the first time, so after reading (1.6), for example, the symbol table might contain the entries shown in Fig. 1.12. In that figure, we have assumed that a word, consisting of four bytes, is set aside for each identifier, and that addresses are assigned starting from byte 0.

IDENTIFIER	ADDRESS
a	0
b	4

Fig. 1.12. An assembler's symbol table with identifiers of (1.6).

In the second pass, the assembler scans the input again. This time, it translates each operation code into the sequence of bits representing that operation in machine language, and it translates each identifier representing a location into the address given for that identifier in the symbol table.

The output of the second pass is usually *relocatable* machine code, meaning that it can be loaded starting at any location L in memory; i.e., if L is added to all addresses in the code, then all references will be correct. Thus, the output of the assembler must distinguish those portions of instructions that refer to addresses that can be relocated.

Example 1.3. The following is a hypothetical machine code into which the assembly instructions (1.6) might be translated.

```
0001 01 00 00000000 *
0011 01 10 00000010                    (1.7)
0010 01 00 00000100 *
```

We envision a tiny instruction word, in which the first four bits are the instruction code, with 0001, 0010, and 0011 standing for load, store, and add, respectively. By load and store we mean moves from memory into a register and vice versa. The next two bits designate a register, and 01 refers to register 1 in each of the three above instructions. The two bits after that represent a "tag," with 00 standing for the ordinary address mode, where the

last eight bits refer to a memory address. The tag 10 stands for the "immediate" mode, where the last eight bits are taken literally as the operand. This mode appears in the second instruction of (1.7).

We also see in (1.7) a * associated with the first and third instructions. This * represents the *relocation bit* that is associated with each operand in relocatable machine code. Suppose that the address space containing the data is to be loaded starting at location L. The presence of the * means that L must be added to the address of the instruction. Thus, if L = 00001111, i.e., 15, then a and b would be at locations 15 and 19, respectively, and the instructions of (1.7) would appear as

```
0001 01 00 00001111
0011 01 10 00000010                    (1.8)
0010 01 00 00010011
```

in *absolute*, or unrelocatable, machine code. Note that there is no * associated with the second instruction in (1.7), so L has not been added to its address in (1.8), which is exactly right because the bits represents the constant 2, not the location 2. □

Loaders and Link-Editors

Usually, a program called a *loader* performs the two functions of loading and link-editing. The process of loading consists of taking relocatable machine code, altering the relocatable addresses as discussed in Example 1.3, and placing the altered instructions and data in memory at the proper locations.

The link-editor allows us to make a single program from several files of relocatable machine code. These files may have been the result of several different compilations, and one or more may be library files of routines provided by the system and available to any program that needs them.

If the files are to be used together in a useful way, there may be some *external* references, in which the code of one file refers to a location in another file. This reference may be to a data location defined in one file and used in another, or it may be to the entry point of a procedure that appears in the code for one file and is called from another file. The relocatable machine code file must retain the information in the symbol table for each data location or instruction label that is referred to externally. If we do not know in advance what might be referred to, we in effect must include the entire assembler symbol table as part of the relocatable machine code.

For example, the code of (1.7) would be preceded by

```
a    0
b    4
```

If a file loaded with (1.7) referred to b, then that reference would be replaced by 4 plus the offset by which the data locations in file (1.7) were relocated.

1.5 THE GROUPING OF PHASES

The discussion of phases in Section 1.3 deals with the logical organization of a compiler. In an implementation, activities from more than one phase are often grouped together.

Front and Back Ends

Often, the phases are collected into a *front end* and a *back end*. The front end consists of those phases, or parts of phases, that depend primarily on the source language and are largely independent of the target machine. These normally include lexical and syntactic analysis, the creation of the symbol table, semantic analysis, and the generation of intermediate code. A certain amount of code optimization can be done by the front end as well. The front end also includes the error handling that goes along with each of these phases.

The back end includes those portions of the compiler that depend on the target machine, and generally, these portions do not depend on the source language, just the intermediate language. In the back end, we find aspects of the code optimization phase, and we find code generation, along with the necessary error handling and symbol-table operations.

It has become fairly routine to take the front end of a compiler and redo its associated back end to produce a compiler for the same source language on a different machine. If the back end is designed carefully, it may not even be necessary to redesign too much of the back end; this matter is discussed in Chapter 9. It is also tempting to compile several different languages into the same intermediate language and use a common back end for the different front ends, thereby obtaining several compilers for one machine. However, because of subtle differences in the viewpoints of different languages, there has been only limited success in this direction.

Passes

Several phases of compilation are usually implemented in a single *pass* consisting of reading an input file and writing an output file. In practice, there is great variation in the way the phases of a compiler are grouped into passes, so we prefer to organize our discussion of compiling around phases rather than passes. Chapter 12 discusses some representative compilers and mentions the way they have structured the phases into passes.

As we have mentioned, it is common for several phases to be grouped into one pass, and for the activity of these phases to be interleaved during the pass. For example, lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped into one pass. If so, the token stream after lexical analysis may be translated directly into intermediate code. In more detail, we may think of the syntax analyzer as being "in charge." It attempts to discover the grammatical structure on the tokens it sees; it obtains tokens as it needs them, by calling the lexical analyzer to find the next token. As the grammatical structure is discovered, the parser calls the intermediate

code generator to perform semantic analysis and generate a portion of the code. A compiler organized this way is presented in Chapter 2.

Reducing the Number of Passes

It is desirable to have relatively few passes, since it takes time to read and write intermediate files. On the other hand, if we group several phases into one pass, we may be forced to keep the entire program in memory, because one phase may need information in a different order than a previous phase produces it. The internal form of the program may be considerably larger than either the source program or the target program, so this space may not be a trivial matter.

For some phases, grouping into one pass presents few problems. For example, as we mentioned above, the interface between the lexical and syntactic analyzers can often be limited to a single token. On the other hand, it is often very hard to perform code generation until the intermediate representation has been completely generated. For example, languages like PL/I and Algol 68 permit variables to be used before they are declared. We cannot generate the target code for a construct if we do not know the types of variables involved in that construct. Similarly, most languages allow goto's that jump forward in the code. We cannot determine the target address of such a jump until we have seen the intervening source code and generated target code for it.

In some cases, it is possible to leave a blank slot for missing information, and fill in the slot when the information becomes available. In particular, intermediate and target code generation can often be merged into one pass using a technique called "backpatching." While we cannot explain all the details until we have seen intermediate-code generation in Chapter 8, we can illustrate backpatching in terms of an assembler. Recall that in the previous section we discussed a two-pass assembler, where the first pass discovered all the identifiers that represent memory locations and deduced their addresses as they were discovered. Then a second pass substituted addresses for identifiers.

We can combine the action of the passes as follows. On encountering an assembly statement that is a forward reference, say

```
GOTO target
```

we generate a skeletal instruction, with the machine operation code for GOTO and blanks for the address. All instructions with blanks for the address of `target` are kept in a list associated with the symbol-table entry for `target`. The blanks are filled in when we finally encounter an instruction such as

```
target: MOV foobar, R1
```

and determine the value of `target`; it is the address of the current instruction. We then "backpatch," by going down the list for `target` of all the instructions that need its address, substituting the address of `target` for the

blanks in the address fields of those instructions. This approach is easy to implement if the instructions can be kept in memory until all target addresses can be determined.

This approach is a reasonable one for an assembler that can keep all its output in memory. Since the intermediate and final representations of code for an assembler are roughly the same, and surely of approximately the same length, backpatching over the length of the entire assembly program is not infeasible. However, in a compiler, with a space-consuming intermediate code, we may need to be careful about the distance over which backpatching occurs.

1.6 COMPILER-CONSTRUCTION TOOLS

The compiler writer, like any programmer, can profitably use software tools such as debuggers, version managers, profilers, and so on. In Chapter 11, we shall see how some of these tools can be used to implement a compiler. In addition to these software-development tools, other more specialized tools have been developed for helping implement various phases of a compiler. We mention them briefly in this section; they are covered in detail in the appropriate chapters.

Shortly after the first compilers were written, systems to help with the compiler-writing process appeared. These systems have often been referred to as *compiler-compilers*, *compiler-generators*, or *translator-writing systems*. Largely, they are oriented around a particular model of languages, and they are most suitable for generating compilers of languages similar to the model.

For example, it is tempting to assume that lexical analyzers for all languages are essentially the same, except for the particular keywords and signs recognized. Many compiler-compilers do in fact produce fixed lexical analysis routines for use in the generated compiler. These routines differ only in the list of keywords recognized, and this list is all that needs to be supplied by the user. The approach is valid, but may be unworkable if it is required to recognize nonstandard tokens, such as identifiers that may include certain characters other than letters and digits.

Some general tools have been created for the automatic design of specific compiler components. These tools use specialized languages for specifying and implementing the component, and many use algorithms that are quite sophisticated. The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of a compiler. The following is a list of some useful compiler-construction tools:

1. *Parser generators.* These produce syntax analyzers, normally from input that is based on a context-free grammar. In early compilers, syntax analysis consumed not only a large fraction of the running time of a compiler, but a large fraction of the intellectual effort of writing a compiler. This phase is now considered one of the easiest to implement. Many of

the "little languages" used to typeset this book, such as PIC (Kernighan [1982]) and EQN, were implemented in a few days using the parser generator described in Section 4.7. Many parser generators utilize powerful parsing algorithms that are too complex to be carried out by hand.

2. *Scanner generators.* These automatically generate lexical analyzers, normally from a specification based on regular expressions, discussed in Chapter 3. The basic organization of the resulting lexical analyzer is in effect a finite automaton. A typical scanner generator and its implementation are discussed in Sections 3.5 and 3.8.

3. *Syntax-directed translation engines.* These produce collections of routines that walk the parse tree, such as Fig. 1.4, generating intermediate code. The basic idea is that one or more "translations" are associated with each node of the parse tree, and each translation is defined in terms of translations at its neighbor nodes in the tree. Such engines are discussed in Chapter 5.

4. *Automatic code generators.* Such a tool takes a collection of rules that define the translation of each operation of the intermediate language into the machine language for the target machine. The rules must include sufficient detail that we can handle the different possible access methods for data; e.g., variables may be in registers, in a fixed (static) location in memory, or may be allocated a position on a stack. The basic technique is "template matching." The intermediate code statements are replaced by "templates" that represent sequences of machine instructions, in such a way that the assumptions about storage of variables match from template to template. Since there are usually many options regarding where variables are to be placed (e.g., in one of several registers or in memory), there are many possible ways to "tile" intermediate code with a given set of templates, and it is necessary to select a good tiling without a combinatorial explosion in running time of the compiler. Tools of this nature are covered in Chapter 9.

5. *Data-flow engines.* Much of the information needed to perform good code optimization involves "data-flow analysis," the gathering of information about how values are transmitted from one part of a program to each other part. Different tasks of this nature can be performed by essentially the same routine, with the user supplying details of the relationship between intermediate code statements and the information being gathered. A tool of this nature is described in Section 10.11.

BIBLIOGRAPHIC NOTES

Writing in 1962 on the history of compiler writing, Knuth [1962] observed that, "In this field there has been an unusual amount of parallel discovery of the same technique by people working independently." He continued by observing that several individuals had in fact discovered "various aspects of a

technique, and it has been polished up through the years into a very pretty algorithm, which none of the originators fully realized." Ascribing credit for techniques remains a perilous task; the bibliographic notes in this book are intended merely as an aid for further study of the literature.

Historical notes on the development of programming languages and compilers until the arrival of Fortran may be found in Knuth and Trabb Pardo [1977]. Wexelblat [1981] contains historical recollections about several programming languages by participants in their development.

Some fundamental early papers on compiling have been collected in Rosen [1967] and Pollack [1972]. The January 1961 issue of the *Communications of the ACM* provides a snapshot of the state of compiler writing at the time. A detailed account of an early Algol 60 compiler is given by Randell and Russell [1964].

Beginning in the early 1960's with the study of syntax, theoretical studies have had a profound influence on the development of compiler technology, perhaps, at least as much influence as in any other area of computer science. The fascination with syntax has long since waned, but compiling as a whole continues to be the subject of lively research. The fruits of this research will become evident when we examine compiling in more detail in the following chapters.

CHAPTER 2

A Simple One-Pass Compiler

This chapter is an introduction to the material in Chapters 3 through 8 of this book. It presents a number of basic compiling techniques that are illustrated by developing a working C program that translates infix expressions into postfix form. Here, the emphasis is on the front end of a compiler, that is, on lexical analysis, parsing, and intermediate code generation. Chapters 9 and 10 cover code generation and optimization.

2.1 OVERVIEW

A programming language can be defined by describing what its programs look like (the *syntax* of the language) and what its programs mean (the *semantics* of the language). For specifying the syntax of a language, we present a widely used notation, called context-free grammars or BNF (for Backus-Naur Form). With the notations currently available, the semantics of a language is much more difficult to describe than the syntax. Consequently, for specifying the semantics of a language we shall use informal descriptions and suggestive examples.

Besides specifying the syntax of a language, a context-free grammar can be used to help guide the translation of programs. A grammar-oriented compiling technique, known as *syntax-directed translation*, is very helpful for organizing a compiler front end and will be used extensively throughout this chapter.

In the course of discussing syntax-directed translation, we shall construct a compiler that translates infix expressions into postfix form, a notation in which the operators appear after their operands. For example, the postfix form of the expression `9-5+2` is `95-2+`. Postfix notation can be converted directly into code for a computer that performs all its computations using a stack. We begin by constructing a simple program to translate expressions consisting of digits separated by plus and minus signs into postfix form. As the basic ideas become clear, we extend the program to handle more general programming language constructs. Each of our translators is formed by systematically extending the previous one.

In our compiler, the *lexical analyzer* converts the stream of input characters into a stream of tokens that becomes the input to the following phase, as shown in Fig. 2.1. The "syntax-directed translator" in the figure is a combination of a syntax analyzer and an intermediate-code generator. One reason for starting with expressions consisting of digits and operators is to make lexical analysis initially very easy; each input character forms a single token. Later, we extend the language to include lexical constructs such as numbers, identifiers, and keywords. For this extended language we shall construct a lexical analyzer that collects consecutive input characters into the appropriate tokens. The construction of lexical analyzers will be discussed in detail in Chapter 3.

Fig. 2.1. Structure of our compiler front end.

2.2 SYNTAX DEFINITION

In this section, we introduce a notation, called a context-free grammar (grammar, for short), for specifying the syntax of a language. It will be used throughout this book as part of the specification of the front end of a compiler.

A grammar naturally describes the hierarchical structure of many programming language constructs. For example, an if-else statement in C has the form

if (expression) statement **else** statement

That is, the statement is the concatenation of the keyword **if**, an opening parenthesis, an expression, a closing parenthesis, a statement, the keyword **else**, and another statement. (In C, there is no keyword **then**.) Using the variable *expr* to denote an expression and the variable *stmt* to denote a statement, this structuring rule can be expressed as

$$stmt \rightarrow \textbf{if} \ (\ expr\ )\ stmt\ \textbf{else}\ stmt \qquad (2.1)$$

in which the arrow may be read as "can have the form." Such a rule is called a *production*. In a production lexical elements like the keyword **if** and the parentheses are called *tokens*. Variables like *expr* and *stmt* represent sequences of tokens and are called *nonterminals*.

A *context-free grammar* has four components:

1. A set of tokens, known as *terminal* symbols.

2. A set of nonterminals.

3. A set of productions where each production consists of a nonterminal, called the *left side* of the production, an arrow, and a sequence of tokens and/or nonterminals, called the *right side* of the production.

4. A designation of one of the nonterminals as the *start* symbol.

We follow the convention of specifying grammars by listing their productions, with the productions for the start symbol listed first. We assume that digits, signs such as <=, and boldface strings such as **while** are terminals. An italicized name is a nonterminal and any nonitalicized name or symbol may be assumed to be a token.[1] For notational convenience, productions with the same nonterminal on the left can have their right sides grouped, with the alternative right sides separated by the symbol |, which we read as "or."

Example 2.1. Several examples in this chapter use expressions consisting of digits and plus and minus signs, e.g., 9-5+2, 3-1, and 7. Since a plus or minus sign must appear between two digits, we refer to such expressions as "lists of digits separated by plus or minus signs." The following grammar describes the syntax of these expressions. The productions are:

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \tag{2.2}$$
$$\textit{list} \rightarrow \textit{list} - \textit{digit} \tag{2.3}$$
$$\textit{list} \rightarrow \textit{digit} \tag{2.4}$$
$$\textit{digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \tag{2.5}$$

The right sides of the three productions with nonterminal *list* on the left side can equivalently be grouped:

$$\textit{list} \rightarrow \textit{list} + \textit{digit} \mid \textit{list} - \textit{digit} \mid \textit{digit}$$

According to our conventions, the tokens of the grammar are the symbols

$$+ \quad - \quad 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9$$

The nonterminals are the italicized names *list* and *digit*, with *list* being the starting nonterminal because its productions are given first. □

We say a production is *for* a nonterminal if the nonterminal appears on the left side of the production. A string of tokens is a sequence of zero or more tokens. The string containing zero tokens, written as ϵ, is called the *empty* string.

A grammar derives strings by beginning with the start symbol and repeatedly replacing a nonterminal by the right side of a production for that

[1] Individual italic letters will be used for additional purposes when grammars are studied in detail in Chapter 4. For example, we shall use X, Y, and Z to talk about a symbol that is either a token or a nonterminal. However, any italicized name containing two or more characters will continue to represent a nonterminal.

nonterminal. The token strings that can be derived from the start symbol form the *language* defined by the grammar.

Example 2.2. The language defined by the grammar of Example 2.1 consists of lists of digits separated by plus and minus signs.

The ten productions for the nonterminal *digit* allow it to stand for any of the tokens 0, 1, . . . , 9. From production (2.4), a single digit by itself is a list. Productions (2.2) and (2.3) express the fact that if we take any list and follow it by a plus or minus sign and then another digit we have a new list.

It turns out that productions (2.2) to (2.5) are all we need to define the language we are interested in. For example, we can deduce that 9-5+2 is a *list* as follows.

a) 9 is a *list* by production (2.4), since 9 is a *digit*.

b) 9-5 is a *list* by production (2.3), since 9 is a *list* and 5 is a *digit*.

c) 9-5+2 is a *list* by production (2.2), since 9-5 is a *list* and 2 is a *digit*.

This reasoning is illustrated by the tree in Fig. 2.2. Each node in the tree is labeled by a grammar symbol. An interior node and its children correspond to a production; the interior node corresponds to the left side of the production, the children to the right side. Such trees are called parse trees and are discussed below. □

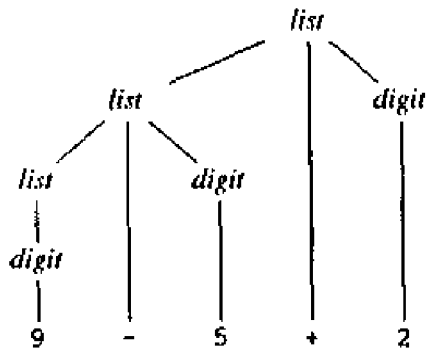


Fig. 2.2. Parse tree for 9-5+2 according to the grammar in Example 2.1.

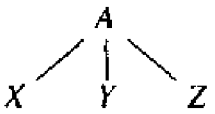
Example 2.3. A somewhat different sort of list is the sequence of statements separated by semicolons found in Pascal begin-end blocks. One nuance of such lists is that an empty list of statements may be found between the tokens **begin** and **end**. We may start to develop a grammar for begin-end blocks by including the productions:

$$
\begin{aligned}
\mathit{block} &\rightarrow \textbf{begin}\ \mathit{opt\_stmts}\ \textbf{end} \\
\mathit{opt\_stmts} &\rightarrow \mathit{stmt\_list} \mid \epsilon \\
\mathit{stmt\_list} &\rightarrow \mathit{stmt\_list}\ ;\ \mathit{stmt} \mid \mathit{stmt}
\end{aligned}
$$

Note that the second possible right side for *opt_stmts* ("optional statement list") is ϵ, which stands for the empty string of symbols. That is, *opt_stmts* can be replaced by the empty string, so a *block* can consist of the two-token string **begin end**. Notice that the productions for *stmt_list* are analogous to those for *list* in Example 2.1, with semicolon in place of the arithmetic operator and *stmt* in place of *digit*. We have not shown the productions for *stmt*. Shortly, we shall discuss the appropriate productions for the various kinds of statements, such as if-statements, assignment statements, and so on. □

Parse Trees

A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. If nonterminal A has a production $A \rightarrow XYZ$, then a parse tree may have an interior node labeled A with three children labeled X, Y, and Z, from left to right:



Formally, given a context-free grammar, a *parse tree* is a tree with the following properties:

1. The root is labeled by the start symbol.
2. Each leaf is labeled by a token or by ϵ.
3. Each interior node is labeled by a nonterminal.
4. If A is the nonterminal labeling some interior node and $X_1, X_2, \ldots, X_n$ are the labels of the children of that node from left to right, then $A \rightarrow X_1 X_2 \cdots X_n$ is a production. Here, $X_1, X_2, \ldots, X_n$ stand for a symbol that is either a terminal or a nonterminal. As a special case, if $A \rightarrow \epsilon$ then a node labeled A may have a single child labeled ϵ.

Example 2.4. In Fig. 2.2, the root is labeled *list*, the start symbol of the grammar in Example 2.1. The children of the root are labeled, from left to right, *list*, +, and *digit*. Note that

list → *list* + *digit*

is a production in the grammar of Example 2.1. The same pattern with - is repeated at the left child of the root, and the three nodes labeled *digit* each have one child that is labeled by a digit. □

The leaves of a parse tree read from left to right form the *yield* of the tree, which is the string *generated* or *derived* from the nonterminal at the root of the parse tree. In Fig. 2.2, the generated string is 9-5+2. In that figure, all the leaves are shown at the bottom level. Henceforth, we shall not necessarily

line up the leaves in this way. Any tree imparts a natural left-to-right order to its leaves, based on the idea that if a and b are two children with the same parent, and a is to the left of b, then all descendants of a are to the left of descendants of b.

Another definition of the language generated by a grammar is as the set of strings that can be generated by some parse tree. The process of finding a parse tree for a given string of tokens is called *parsing* that string.

Ambiguity

We have to be careful in talking about *the* structure of a string according to a grammar. While it is clear that each parse tree derives exactly the string read off its leaves, a grammar can have more than one parse tree generating a given string of tokens. Such a grammar is said to be *ambiguous*. To show that a grammar is ambiguous, all we need to do is find a token string that has more than one parse tree. Since a string with more than one parse tree usually has more than one meaning, for compiling applications we need to design unambiguous grammars, or to use ambiguous grammars with additional rules to resolve the ambiguities.

Example 2.5. Suppose we did not distinguish between digits and lists as in Example 2.1. We could have written the grammar

string → *string* + *string* | *string* - *string* | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Merging the notion of *digit* and *list* into the nonterminal *string* makes superficial sense, because a single *digit* is a special case of a *list*.

However, Fig. 2.3 shows that an expression like 9-5+2 now has more than one parse tree. The two trees for 9-5+2 correspond to the two ways of parenthesizing the expression: (9-5)+2 and 9-(5+2). This second parenthesization gives the expression the value 2 rather than the customary value 6. The grammar of Example 2.1 did not permit this interpretation. □

Associativity of Operators

By convention, 9+5+2 is equivalent to (9+5)+2 and 9-5-2 is equivalent to (9-5)-2. When an operand like 5 has operators to its left and right, conventions are needed for deciding which operator takes that operand. We say that the operator + *associates to the left* because an operand with plus signs on both sides of it is taken by the operator to its left. In most programming languages the four arithmetic operators, addition, subtraction, multiplication, and division are left associative.

Some common operators such as exponentiation are right associative. As another example, the assignment operator = in C is right associative; in C, the expression a=b=c is treated in the same way as the expression a=(b=c).

Strings like a=b=c with a right-associative operator are generated by the following grammar:

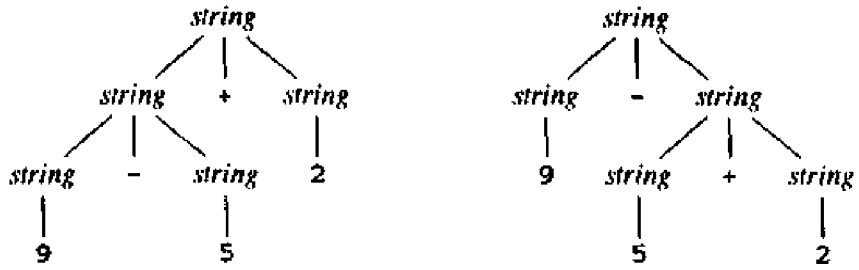


Fig. 2.3. Two parse trees for 9-5+2.

right → *letter* = *right* | *letter*
letter → a | b | · · · | z

The contrast between a parse tree for a left-associative operator like - and a parse tree for a right-associative operator like = is shown by Fig. 2.4. Note that the parse tree for 9-5-2 grows down towards the left, whereas the parse tree for a=b=c grows down towards the right.

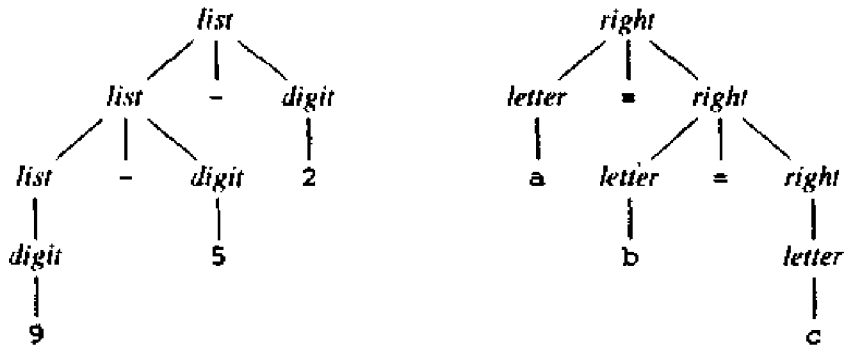


Fig. 2.4. Parse trees for left- and right-associative operators.

Precedence of Operators

Consider the expression 9+5*2. There are two possible interpretations of this expression: (9+5)*2 or 9+(5*2). The associativity of + and * do not resolve this ambiguity. For this reason, we need to know the relative precedence of operators when more than one kind of operator is present.

We say that * has *higher precedence* than + if * takes its operands before + does. In ordinary arithmetic, multiplication and division have higher precedence than addition and subtraction. Therefore, 5 is taken by * in both 9+5*2 and 9*5+2; i.e., the expressions are equivalent to 9+(5*2) and (9*5)+2, respectively.

Syntax of expressions. A grammar for arithmetic expressions can be

constructed from a table showing the associativity and precedence of operators. We start with the four common arithmetic operators and a precedence table, showing the operators in order of increasing precedence with operators at the same precedence level on the same line:

left associative: + -
left associative: * /

We create two nonterminals *expr* and *term* for the two levels of precedence, and an extra nonterminal *factor* for generating basic units in expressions. The basic units in expressions are presently digits and parenthesized expressions.

factor → **digit** | (*expr*)

Now consider the binary operators, * and /, that have the highest precedence. Since these operators associate to the left, the productions are similar to those for lists that associate to the left.

term → *term* * *factor*
| *term* / *factor*
| *factor*

Similarly, *expr* generates lists of terms separated by the additive operators.

expr → *expr* + *term*
| *expr* - *term*
| *term*

The resulting grammar is therefore

expr → *expr* + *term* | *expr* - *term* | *term*
term → *term* * *factor* | *term* / *factor* | *factor*
factor → **digit** | (*expr*)

This grammar treats an expression as a list of terms separated by either + or - signs, and a term as a list of factors separated by * or / signs. Notice that any parenthesized expression is a factor, so with parentheses we can develop expressions that have arbitrarily deep nesting (and also arbitrarily deep trees).

Syntax of statements. Keywords allow us to recognize statements in most languages. All Pascal statements begin with a keyword except assignments and procedure calls. Some Pascal statements are defined by the following (ambiguous) grammar in which the token **id** represents an identifier.

stmt → **id** := *expr*
| **if** *expr* **then** *stmt*
| **if** *expr* **then** *stmt* **else** *stmt*
| **while** *expr* **do** *stmt*
| **begin** *opt_stmts* **end**

The nonterminal *opt_stmts* generates a possibly empty list of statements separated by semicolons using the productions in Example 2.3.

2.3 SYNTAX-DIRECTED TRANSLATION

To translate a programming language construct, a compiler may need to keep track of many quantities besides the code generated for the construct. For example, the compiler may need to know the type of the construct, or the location of the first instruction in the target code, or the number of instructions generated. We therefore talk abstractly about *attributes* associated with constructs. An attribute may represent any quantity, e.g., a type, a string, a memory location, or whatever.

In this section, we present a formalism called a syntax-directed definition for specifying translations for programming language constructs. A syntax-directed definition specifies the translation of a construct in terms of attributes associated with its syntactic components. In later chapters, syntax-directed definitions are used to specify many of the translations that take place in the front end of a compiler.

We also introduce a more procedural notation, called a translation scheme, for specifying translations. Throughout this chapter, we use translation schemes for translating infix expressions into postfix notation. A more detailed discussion of syntax-directed definitions and their implementation is contained in Chapter 5.

Postfix Notation

The *postfix notation* for an expression E can be defined inductively as follows:

1. If E is a variable or constant, then the postfix notation for E is E itself.
2. If E is an expression of the form E_1 *op* E_2, where *op* is any binary operator, then the postfix notation for E is E_1' E_2' *op*, where E_1' and E_2' are the postfix notations for E_1 and E_2, respectively.
3. If E is an expression of the form (E_1), then the postfix notation for E_1 is also the postfix notation for E.

No parentheses are needed in postfix notation because the position and arity (number of arguments) of the operators permits only one decoding of a postfix expression. For example, the postfix notation for `(9-5)+2` is `95-2+` and the postfix notation for `9-(5+2)` is `952+-`.

Syntax-Directed Definitions

A *syntax-directed definition* uses a context-free grammar to specify the syntactic structure of the input. With each grammar symbol, it associates a set of attributes, and with each production, a set of *semantic rules* for computing values of the attributes associated with the symbols appearing in that production. The grammar and the set of semantic rules constitute the syntax-directed definition.

A translation is an input-output mapping. The output for each input x is specified in the following manner. First, construct a parse tree for x. Suppose

a node n in the parse tree is labeled by the grammar symbol X. We write $X.a$ to denote the value of attribute a of X at that node. The value of $X.a$ at n is computed using the semantic rule for attribute a associated with the X-production used at node n. A parse tree showing the attribute values at each node is called an *annotated* parse tree.

Synthesized Attributes

An attribute is said to be *synthesized* if its value at a parse-tree node is determined from attribute values at the children of the node. Synthesized attributes have the desirable property that they can be evaluated during a single bottom-up traversal of the parse tree. In this chapter, only synthesized attributes are used; "inherited" attributes are considered in Chapter 5.

Example 2.6. A syntax-directed definition for translating expressions consisting of digits separated by plus or minus signs into postfix notation is shown in Fig. 2.5. Associated with each nonterminal is a string-valued attribute t that represents the postfix notation for the expression generated by that nonterminal in a parse tree.

PRODUCTION	SEMANTIC RULE
$expr \rightarrow expr_1 + term$	$expr.t := expr_1.t \parallel term.t \parallel '+'$
$expr \rightarrow expr_1 - term$	$expr.t := expr_1.t \parallel term.t \parallel '-'$
$expr \rightarrow term$	$expr.t := term.t$
$term \rightarrow 0$	$term.t := '0'$
$term \rightarrow 1$	$term.t := '1'$
...	...
$term \rightarrow 9$	$term.t := '9'$

Fig. 2.5. Syntax-directed definition for infix to postfix translation.

The postfix form of a digit is the digit itself; e.g., the semantic rule associated with the production $term \rightarrow 9$ defines $term.t$ to be 9 whenever this production is used at a node in a parse tree. When the production $expr \rightarrow term$ is applied, the value of $term.t$ becomes the value of $expr.t$.

The production $expr \rightarrow expr_1 + term$ derives an expression containing a plus operator (the subscript in $expr_1$ distinguishes the instance of $expr$ on the right from that on the left side). The left operand of the plus operator is given by $expr_1$ and the right operand by $term$. The semantic rule

$$expr.t := expr_1.t \parallel term.t \parallel '+'$$

associated with this production defines the value of attribute $expr.t$ by concatenating the postfix forms $expr_1.t$ and $term.t$ of the left and right operands, respectively, and then appending the plus sign. The operator $\parallel$ in semantic

rules represents string concatenation.

Figure 2.6 contains the annotated parse tree corresponding to the tree of Fig. 2.2. The value of the t-attribute at each node has been computed using the semantic rule associated with the production used at that node. The value of the attribute at the root is the postfix notation for the string generated by the parse tree. □

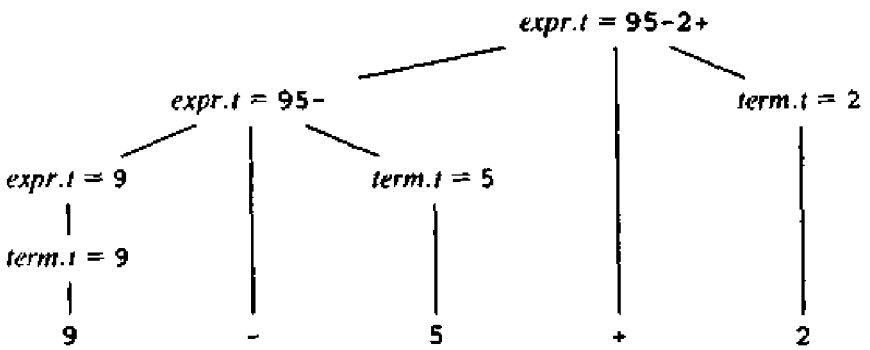


Fig. 2.6. Attribute values at nodes in a parse tree.

Example 2.7. Suppose a robot can be instructed to move one step east, north, west, or south from its current position. A sequence of such instructions is generated by the following grammar:

$$seq \rightarrow seq\ \ instr \mid \textbf{begin}$$
$$instr \rightarrow \textbf{east} \mid \textbf{north} \mid \textbf{west} \mid \textbf{south}$$

Changes in the position of the robot on input

begin west south east east east north north

are shown in Fig. 2.7.

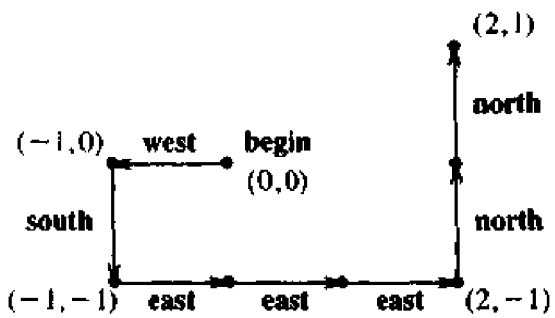


Fig. 2.7. Keeping track of a robot's position.

In the figure, a position is marked by a pair (x,y), where x and y represent the number of steps to the east and north, respectively, from the starting

position. (If x is negative, then the robot is to the west of the starting position; similarly, if y is negative, then the robot is to the south of the starting position.)

Let us construct a syntax-directed definition to translate an instruction sequence into a robot position. We shall use two attributes, *seq.x* and *seq.y*, to keep track of the position resulting from an instruction sequence generated by the nonterminal *seq*. Initially, *seq* generates **begin**, and *seq.x* and *seq.y* are both initialized to 0, as shown at the leftmost interior node of the parse tree for **begin west south** shown in Fig. 2.8.

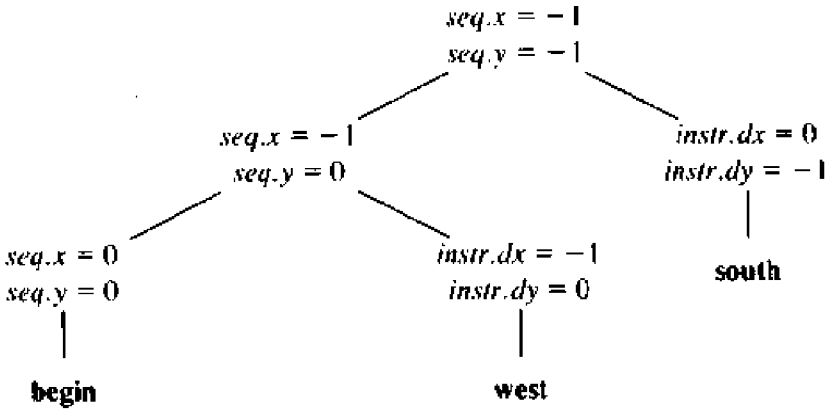


Fig. 2.8. Annotated parse tree for **begin west south**.

The change in position due to an individual instruction derived from *instr* is given by attributes *instr.dx* and *instr.dy*. For example, if *instr* derives **west**, then *instr.dx* = −1 and *instr.dy* = 0. Suppose a sequence *seq* is formed by following a sequence seq_1 by a new instruction *instr*. The new position of the robot is then given by the rules

$$seq.x := seq_1.x + instr.dx$$
$$seq.y := seq_1.y + instr.dy$$

A syntax-directed definition for translating an instruction sequence into a robot position is shown in Fig. 2.9. □

Depth-First Traversals

A syntax-directed definition does not impose any specific order for the evaluation of attributes on a parse tree; any evaluation order that computes an attribute a after all the other attributes that a depends on is acceptable. In general, we may have to evaluate some attributes when a node is first reached during a walk of the parse tree, others after all its children have been visited, or at some point in between visits to the children of the node. Suitable evaluation orders are discussed in more detail in Chapter 5.

The translations in this chapter can all be implemented by evaluating the semantic rules for the attributes in a parse tree in a predetermined order. A *traversal* of a tree starts at the root and visits each node of the tree in some

PRODUCTION	SEMANTIC RULES
seq → **begin**	*seq.x* := 0 *seq.y* := 0
seq → seq_1 *instr*	*seq.x* := $seq_1.x$ + *instr.dx* *seq.y* := $seq_1.y$ + *instr.dy*
instr → **east**	*instr.dx* := 1 *instr.dy* := 0
instr → **north**	*instr.dx* := 0 *instr.dy* := 1
instr → **west**	*instr.dx* := −1 *instr.dy* := 0
instr → **south**	*instr.dx* := 0 *instr.dy* := −1

Fig. 2.9. Syntax-directed definition of the robot's position.

order. In this chapter, semantic rules will be evaluated using the depth-first traversal defined in Fig. 2.10. It starts at the root and recursively visits the children of each node in left-to-right order, as shown in Fig. 2.11. The semantic rules at a given node are evaluated once all descendants of that node have been visited. It is called "depth-first" because it visits an unvisited child of a node whenever it can, so it tries to visit nodes as far away from the root as quickly as it can.

```
procedure visit(n : node);
begin
    for each child m of n, from left to right do
        visit(m);
    evaluate semantic rules at node n
end
```

Fig. 2.10. A depth-first traversal of a tree.

Translation Schemes

In the remainder of this chapter, we use a procedural specification for defining a translation. A *translation scheme* is a context-free grammar in which program fragments called *semantic actions* are embedded within the right sides of productions. A translation scheme is like a syntax-directed definition, except that the order of evaluation of the semantic rules is explicitly shown. The

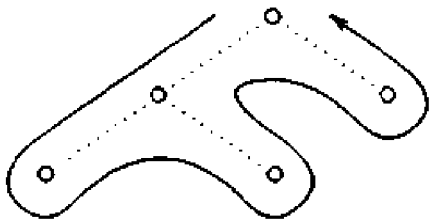


Fig. 2.11. Example of a depth-first traversal of a tree.

position at which an action is to be executed is shown by enclosing it between braces and writing it within the right side of a production, as in

$$rest \rightarrow + \; term \; \{ print('+') \} \; rest_1$$

A translation scheme generates an output for each sentence x generated by the underlying grammar by executing the actions in the order they appear during a depth-first traversal of a parse tree for x. For example, consider a parse tree with a node labeled *rest* representing this production. The action $\{ print('+') \}$ will be performed after the subtree for *term* is traversed but before the child for $rest_1$ is visited.

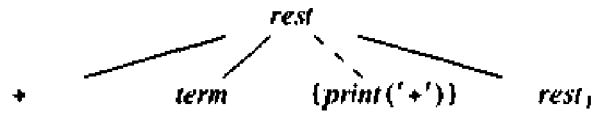


Fig. 2.12. An extra leaf is constructed for a semantic action.

When drawing a parse tree for a translation scheme, we indicate an action by constructing for it an extra child, connected by a dashed line to the node for its production. For example, the portion of the parse tree for the above production and action is drawn as in Fig. 2.12. The node for a semantic action has no children, so the action is performed when that node is first seen.

Emitting a Translation

In this chapter, the semantic actions in translations schemes will write the output of a translation into a file, a string or character at a time. For example, we translate `9-5+2` into `95-2+` by printing each character in `9-5+2` exactly once, without using any storage for the translation of subexpressions. When the output is created incrementally in this fashion, the order in which the characters are printed is important.

Notice that the syntax-directed definitions mentioned so far have the following important property: the string representing the translation of the nonterminal on the left side of each production is the concatenation of the translations

of the nonterminals on the right, in the same order as in the production, with some additional strings (perhaps none) interleaved. A syntax-directed definition with this property is termed *simple*. For example, consider the first production and semantic rule from the syntax-directed definition of Fig. 2.5:

PRODUCTION	SEMANTIC RULE	
$expr \rightarrow expr_1 + term$	$expr.t := expr_1.t \parallel term.t \parallel '+'$	(2.6)

Here the translation $expr.t$ is the concatenation of the translations of $expr_1$ and *term*, followed by the symbol +. Notice that $expr_1$ appears before *term* on the right side of the production.

An additional string appears between $term.t$ and $rest_1.t$ in

PRODUCTION	SEMANTIC RULE	
$rest \rightarrow + term\ rest_1$	$rest.t := term.t \parallel '+' \parallel rest_1.t$	(2.7)

but, again, the nonterminal *term* appears before $rest_1$ on the right side.

Simple syntax-directed definitions can be implemented with translation schemes in which actions print the additional strings in the order they appear in the definition. The actions in the following productions print the additional strings in (2.6) and (2.7), respectively:

$$expr \rightarrow expr_1 + term \ \{ print('+') \}$$
$$rest \rightarrow + term \ \{ print('+') \} \ rest_1$$

Example 2.8. Figure 2.5 contained a simple definition for translating expressions into postfix form. A translation scheme derived from this definition is given in Fig. 2.13 and a parse tree with actions for `9-5+2` is shown in Fig. 2.14. Note that although Figures 2.6 and 2.14 represent the same input-output mapping, the translation in the two cases is constructed differently; Fig. 2.6 attaches the output to the root of the parse tree, while Fig. 2.14 prints the output incrementally.

$$\begin{array}{lcll} expr & \rightarrow & expr + term & \{ print('+') \} \\ expr & \rightarrow & expr - term & \{ print('-') \} \\ expr & \rightarrow & term & \\ term & \rightarrow & 0 & \{ print('0') \} \\ term & \rightarrow & 1 & \{ print('1') \} \\ & \cdots & & \\ term & \rightarrow & 9 & \{ print('9') \} \end{array}$$

Fig. 2.13. Actions translating expressions into postfix notation.

The root of Fig. 2.14 represents the first production in Fig. 2.13. In a depth-first traversal, we first perform all the actions in the subtree for the left operand *expr* when we traverse the leftmost subtree of the root. We then visit the leaf + at which there is no action. We next perform the actions in the

subtree for the right operand *term* and, finally, the semantic action { *print*('+') } at the extra node.

Since the productions for *term* have only a digit on the right side, that digit is printed by the actions for the productions. No output is necessary for the production *expr* → *term*, and only the operator needs to be printed in the action for the first two productions. When executed during a depth-first traversal of the parse tree, the actions in Fig. 2.14 print 95-2+. □

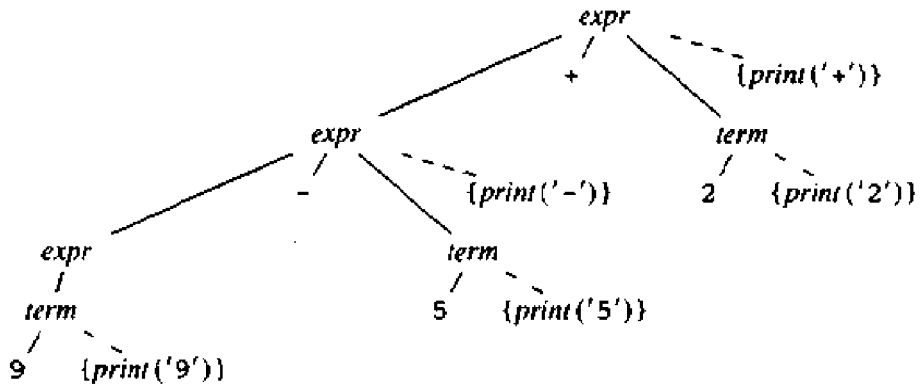


Fig. 2.14. Actions translating 9-5+2 into 95-2+.

As a general rule, most parsing methods process their input from left to right in a "greedy" fashion; that is, they construct as much of a parse tree as possible before reading the next input token. In a simple translation scheme (one derived from a simple syntax-directed definition), actions are also done in a left-to-right order. Therefore, to implement a simple translation scheme we can execute the semantic actions while we parse; it is not necessary to construct the parse tree at all.

2.4 PARSING

Parsing is the process of determining if a string of tokens can be generated by a grammar. In discussing this problem, it is helpful to think of a parse tree being constructed, even though a compiler may not actually construct such a tree. However, a parser must be capable of constructing the tree, or else the translation cannot be guaranteed correct.

This section introduces a parsing method that can be applied to construct syntax-directed translators. A complete C program, implementing the translation scheme of Fig. 2.13, appears in the next section. A viable alternative is to use a software tool to generate a translator directly from a translation scheme. See Section 4.9 for the description of such a tool; it can implement the translation scheme of Fig. 2.13 without modification.

A parser can be constructed for any grammar. Grammars used in practice, however, have a special form. For any context-free grammar there is a parser that takes at most $O(n^3)$ time to parse a string of n tokens. But cubic time is

too expensive. Given a programming language, we can generally construct a grammar that can be parsed quickly. Linear algorithms suffice to parse essentially all languages that arise in practice. Programming language parsers almost always make a single left-to-right scan over the input, looking ahead one token at a time.

Most parsing methods fall into one of two classes, called the *top-down* and *bottom-up* methods. These terms refer to the order in which nodes in the parse tree are constructed. In the former, construction starts at the root and proceeds towards the leaves, while, in the latter, construction starts at the leaves and proceeds towards the root. The popularity of top-down parsers is due to the fact that efficient parsers can be constructed more easily by hand using top-down methods. Bottom-up parsing, however, can handle a larger class of grammars and translation schemes, so software tools for generating parsers directly from grammars have tended to use bottom-up methods.

Top-Down Parsing

We introduce top-down parsing by considering a grammar that is well-suited for this class of methods. Later in this section, we consider the construction of top-down parsers in general. The following grammar generates a subset of the types of Pascal. We use the token **dotdot** for ".." to emphasize that the character sequence is treated as a unit.

$$
\begin{array}{rcl}
\textit{type} & \rightarrow & \textit{simple} \\
 & | & \uparrow \textbf{id} \\
 & | & \textbf{array [} \textit{simple} \textbf{] of } \textit{type} \\
\textit{simple} & \rightarrow & \textbf{integer} \\
 & | & \textbf{char} \\
 & | & \textbf{num dotdot num}
\end{array}
\tag{2.8}
$$

The top-down construction of a parse tree is done by starting with the root, labeled with the starting nonterminal, and repeatedly performing the following two steps (see Fig. 2.15 for an example).

1. At node n, labeled with nonterminal A, select one of the productions for A and construct children at n for the symbols on the right side of the production.
2. Find the next node at which a subtree is to be constructed.

For some grammars, the above steps can be implemented during a single left-to-right scan of the input string. The current token being scanned in the input is frequently referred to as the *lookahead* symbol. Initially, the lookahead symbol is the first, i.e., leftmost, token of the input string. Figure 2.16 illustrates the parsing of the string

array [num dotdot num] of integer

Initially, the token **array** is the lookahead symbol and the known part of the

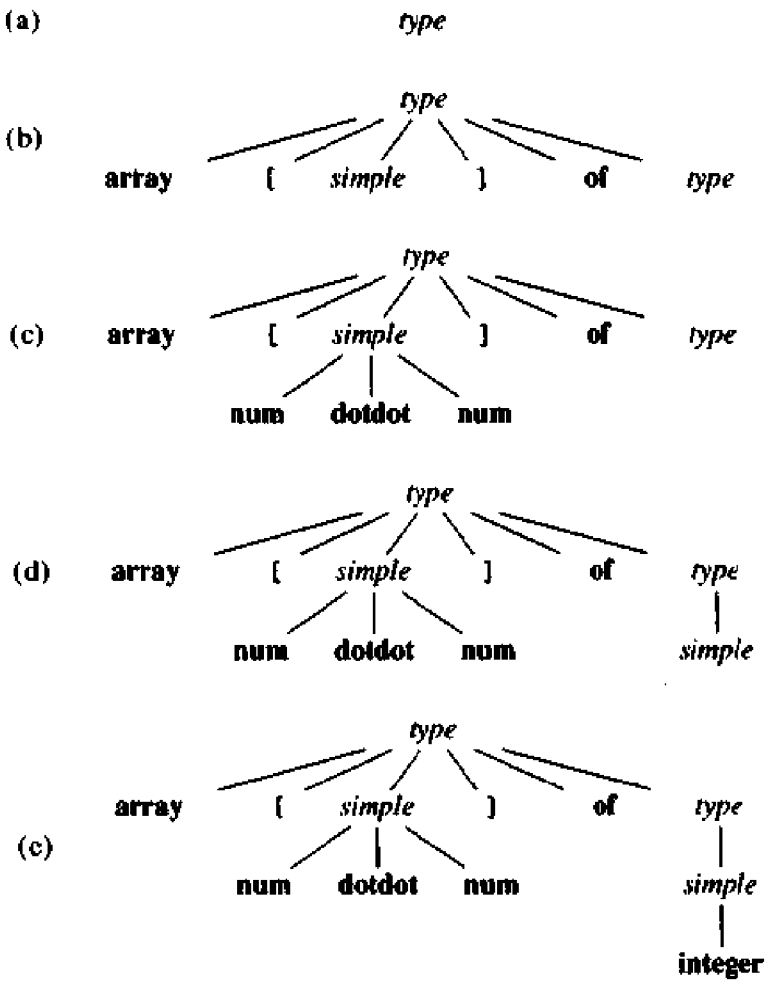


Fig. 2.15. Steps in the top-down construction of a parse tree.

parse tree consists of the root, labeled with the starting nonterminal *type* in Fig. 2.16(a). The objective is to construct the remainder of the parse tree in such a way that the string generated by the parse tree matches the input string.

For a match to occur, nonterminal *type* in Fig. 2.16(a) must derive a string that starts with the lookahead symbol **array**. In grammar (2.8), there is just one production for *type* that can derive such a string, so we select it, and construct the children of the root labeled with the symbols on the right side of the production.

Each of the three snapshots in Fig. 2.16 has arrows marking the lookahead symbol in the input and the node in the parse tree that is being considered. When children are constructed at a node, we next consider the leftmost child. In Fig. 2.16(b), children have just been constructed at the root, and the leftmost child labeled with **array** is being considered.

When the node being considered in the parse tree is for a terminal and the